משימה 5 - הפקודה האחרונה

נושאים: `argc`, `argv`, מחרוזות, מערכים ומבנים, מצביעים, זיכרון דינמי, בדיקות תקינות ושילוב רכיבי תוכנה.

הרצת `./operation_b7` מהתיקייה הראשית יוצרת את `mission5/data/mission5_emergency.b7`. זהו תיק הראיות האישי שלכם; אין לערוך אותו. אם כבר יצרתם תיק חקירה במשימות הקודמות, השתמשו באותם פרטי צוות.

תדריך מבצעי

חצות מתקרבת. הבארשבעינייטור עדיין מסוגל לשבש את מערכות התנועה, הכניסה והתשתיות של באר שבע, וההכנות להפעלתו נמשכות. אם לא נעצור אותו, העיר עלולה לספוג פגיעה קשה.

במשימה הקודמת הצלחתם לתקן את בקר החירום של הבארשבעינייטור ולהחזיר את ממשק ההשמדה העצמית לפעולה. התיקון עבר ללא התרעה, ונראה היה שהנבלים לא הבחינו בהתערבות. אלא שזמן קצר לאחר מכן יורטה השיחה הבאה:

> **ד"ר דופנשמירץ:** פלנקטון! הבקר שלי הפסיק להתרסק!
>
> **פלנקטון:** באמת? מצוין!
>
> **ד"ר דופנשמירץ:** מה תיקנת?
>
> **פלנקטון:** שום דבר. לא נגעתי בו.
>
> **ד"ר דופנשמירץ:** אז איך הוא עובד עכשיו?!
>
> **פלנקטון:** אין לי מושג. אבל יש לי כלל: אם זה עובד, לא נוגעים!
>
> **מורטי הרשע:** תוכנה לא מתקנת את עצמה. מישהו שינה משהו במערכת.
>
> **ד"ר דופנשמירץ:** או שההמצאה שלי כל כך גאונית שהיא למדה לתקן את עצמה!
>
> **מורטי הרשע:** פלנקטון, בדוק את יומן התחזוקה.
>
> **פלנקטון:** בדקתי. אין שום תיעוד לתיקון.
>
> **מורטי הרשע:** וזה בדיוק מה שמדאיג אותי.
>
> **פלנקטון:** אז... עדיין לא נוגעים?

הנבלים הבחינו בשינוי, אך נראה שעדיין לא הבינו מי עומד מאחוריו. בינתיים, הבקר פועל — אלא שאי אפשר להוציא פקודת השמדה ללא הרשאה מתאימה.

בחיפוש נוסף בקובצי התחזוקה נמצאו רישומי בקשות חירום, חלקם פגומים וחלקם שייכים לרכיבים אחרים. במקביל זוהתה פעילות חריגה סביב `PORT_8080`, שכבר הופיע בחקירה הקודמת. לא ברור מה קרה שם, ואם הדבר יכול לסייע לנו להשיג את האישור החסר.

המשימה שלכם היא להכין ולהפעיל את פקודת ההשמדה העצמית לפני שהבארשבעינייטור יופעל. עליכם לפענח את רישומי החירום, להכין בקשה שהבקר יוכל לקבל ולמצוא דרך להשיג את האישור החסר. כל טעות באחת הפעולות עלולה לסכל את ההזדמנות לעצור את המכשיר.

כל אחד מחלקי כלי החירום שתבנו ייבדק בנפרד, אבל רק רצף תקין ושלם יאפשר להוציא את הפקודה לפועל. הממצאים שייחשפו במהלך ההרצות הם חלק מחומר החקירה; קראו אותם לפני מילוי הדוח.

קבצים לעריכה: `mission5/emergency.c` והחלקים המסומנים ב־`mission5/main.c`. אין לשנות את קובצי התשתית שב־`/support/mission5_/..` או את קובץ הראיות האישי.

הרצה ובדיקות

מתוך `/mission5`:

```
make
make test-a
make test-b
make test-c
make test-d
```

כל בדיקת שלב מאמתת רק את הפונקציות המתאימות לו; אפשר להתקדם בין השלבים גם אם חלק אחר עדיין אינו תקין. לאחר שהשלמתם את כל השלבים, הריצו:

```
make run
```

קוד ההשלמה והמשך החקירה יתקבלו **רק מהרצה מלאה ומוצלחת**.

שלב A - העברת הוראות לצוות החירום

עד כה הופעלו תוכניות החקירה בעיקר על קובצי נתונים שנוצרו מראש. כעת נדרש כלי שאפשר להפעיל מהמסוף עם יעד, מצב פעולה ופקודה - בלי לערוך בכל פעם את קוד המקור. אם הוראות ההפעלה אינן שלמות או אינן תואמות למכשיר, אסור להמשיך לשלב האישורים.

המפעיל ישתמש בפקודה מהצורה:

```
./mission5 --device BEERSHEVANATOR --mode emergency --action self-destruct
```

ב־`emergency.h` מוגדר:

```
#define M5_DEVICE_CAPACITY 32
#define M5_MODE_CAPACITY 16
#define M5_ACTION_CAPACITY 32
typedef struct {
    char device[M5_DEVICE_CAPACITY];
    char mode[M5_MODE_CAPACITY];
    char action[M5_ACTION_CAPACITY];
} LaunchOptions;
```

ממשו ב־`emergency.c`:

```
int parse_launch_options(int argc, char *const argv[], LaunchOptions *out);
```

חוזה הפונקציה:

- קבלו בדיוק את שלוש האפשרויות `--mode`, `device` ו־`--action`, כל אחת פעם אחת ועם ערך אחריה. `argv[0]` הוא שם התוכנית ואינו אפשרות.
- הערכים התקינים להפעלה זו הם `emergency`, `BEERSHEVANATOR` ו־`self-destruct`, בהתאמה. סדר האפשרויות יכול להשתנות.
- בהצלחה העתיקו את הערכים לשדות המבנה והחזירו `1`. אין לשמור מצביעים ל־`argv` במקום להעתיק את המחרוזות.
- החזירו `0` עבור ארגומנט חסר, אפשרות לא מוכרת או כפולה, ערך שגוי, מצביע לא תקין או מחרוזת שאינה נכנסת לשדה יחד עם `\0`.
- בכישלון, אם `out` תקין, כל שדותיו יישארו מחרוזות ריקות. אין לקרוא מעבר ל־`argc` או לכתוב מחוץ למערכים.
- אין צורך בהקצאה דינמית.

שינוי סדר האפשרויות לא ישנה את התוצאה:

```
./mission5 --action self-destruct --device BEERSHEVANATOR --mode emergency
```

לעומת זאת, פקודה ללא `--action` חייבת להידחות. לאחר מימוש הפונקציה הריצו `make test-a`. רק הוראה תקינה יכולה לעבור להמשך עיבוד הראיות.

שלב B - רישומי האישורים

הבקר דורש אישור לפעולת חירום. ברישומי המערכת מופיעות בקשות שונות, חלקן אינן קשורות למכשיר וחלקן מכילות נתונים לא תקינים. עליכם לברר איזו רשומה עשויה להתאים לבקשה שלנו, גם אם עדיין לא אושרה.

אל תניחו שהרשומה המתאימה נמצאת בתחילת המערך, ואל תסתמכו על זהות בעל החשבון או על מספר בקשה קבוע מראש.

ב־`emergency.h` מוגדר:

```
typedef struct {
    int request_id;
    int unit_id;
    int approver_id;
    int clearance;
    int approved;
} AuthorizationRecord;
```

`request_id` הוא מזהה הבקשה; `unit_id` הוא מזהה רכיב הבקרה; `approver_id` מזהה את החשבון שמורשה לאשר אותה; `clearance` היא רמת הרשאות החשבון (0–5); ו־`approved` מציין אם הבקשה כבר אושרה (`1`) או שהיא עדיין ממתינה לאישור (`0`).

ממשו:

```
const AuthorizationRecord *find_authorization(
    const AuthorizationRecord *records, size_t count,
    int request_id, int unit_id, int required_clearance);
```

חוזה הפונקציה:

- מצאו את הרשומה הראשונה שעבורה `request_id` ו־`unit_id` תואמים לערכים המבוקשים, `approved == 0` (בקשה ממתינה), ו־`clearance >= required_clearance`. אל תשנו את מצב האישור בשלב זה.
- רשומה תקינה מחייבת מזהי בקשה, רכיב ומאשר חיוביים; `approved` חייב להיות `0` או `1`, ו־`clearance` בטווח 0–5. התעלמו מרשומות פגומות.
- גם מזהי החיפוש חייבים להיות חיוביים ו־`required_clearance` חייב להיות בטווח 0–5. עבור קלט לא תקין או היעדר התאמה החזירו `NULL`.
- החזירו מצביע **לרשומה המקורית** במערך. אין להקצות זיכרון, להחזיר עותק מקומי או לשנות את הרשומות.
- `records == NULL` מותר רק כאשר `count == 0`; בשני המקרים אין התאמה ויש להחזיר `NULL` בבטחה.

רק מזהי הבקשה והרכיב מתוך הראיות האישיות קובעים איזו רשומה רלוונטית. לאחר שהפונקציה תקינה, הריצו `make test-b`. **הפונקציה מאתרת רשומה בלבד; היא אינה משנה את מצבה.**

שלב C - הבקשה חייבת לשרוד

נמצאה רשומה שמתאימה לבקשה? עדיין אי אפשר לשלוח אותה כפי שהיא. כלי החקירה קורא את המידע ממאגר זמני, והבקשה תצטרך להמשיך להתקיים גם לאחר מכן. אם היא רק תצביע למחרוזות ולמערכים שבמאגר, המידע עלול להשתנות או להפוך ללא תקף לפני הביצוע.

לכן עליכם **להכין בקשת חירום עצמאית** שמחזיקה את הנתונים הדרושים לה, גם אחרי שמאגר הראיות המקורי כבר אינו בשימוש.

המבנה מוגדר ב־`emergency.h`:

```
typedef struct {
    int request_id;
    int unit_id;
    AuthorizationRecord authorization;
    char *device;
    char mode[M5_MODE_CAPACITY];
    char action[M5_ACTION_CAPACITY];
    int *steps;
    size_t step_count;
} EmergencyRequest;
```

ממשו:

```
int emergency_request_init(EmergencyRequest *dest,
                           const LaunchOptions *options,
                           const AuthorizationRecord *authorization,
                           const int *steps, size_t step_count);
void emergency_request_free(EmergencyRequest *request);
```

`emergency_request_init` - חוזה הפונקציה:

- `dest` חייבת להיות בקשה ריקה: `device` ו־`steps` הם `NULL`, ויתר השדות מאופסים. אין לדרוס בקשה שמחזיקה הקצאות פעילות.
- `options` חייב להכיל מחרוזות C המסתיימות בגבולות השדות; `device` לא יכולה להיות ריקה. מספר הצעדים `step_count` חייב להיות בטווח 1–8, ו־`steps` חייב להצביע למערך מתאים.
- הקצו בנפרד זיכרון לשם המכשיר ולמערך הצעדים, והעתיקו את שניהם במלואם. העתיקו את `authorization` לפי ערך, **גם כאשר** `approved == 0`, ואת שדות `mode` ו־`action` לתוך המערכים הפנימיים. העבירו את מזהי הבקשה והרכיב מרשומת האישור לשדות המתאימים. אין לאשר את הבקשה בתוך פונקציית האתחול.
- הבקשה המתקבלת חייבת להישאר תקינה גם אם נתוני הקלט משתנים לאחר הקריאה.
- החזירו `1` בהצלחה ו־`0` בכישלון, לרבות קלט לא תקין וכשל הקצאה. בכישלון שחררו כל הקצאה שבוצעה בניסיון והשאירו בקשה ריקה ותקינה.
- אין לשחרר את הקלט המושאל, להשתמש ב־`strdup` או להסתמך על פונקציות שאינן חלק מ־C11. מנעו גלישה בחישובי הקצאה.

`emergency_request_free` - חוזה הפונקציה:

- שחררו את המחרוזת ואת מערך הצעדים שבבעלות הבקשה.
- אפסו את כל השדות כדי שניתן יהיה לשחרר שוב ללא נזק.
- גם `NULL` ובקשה ריקה הם קלט חוקי.

הריצו `make test-c`. בין היתר, הבדיקה תוודא שהבקשה אינה משתנה כאשר מקורות הנתונים משתנים ושאין דליפות זיכרון במקרי כשל.

שלב D - הפקודה האחרונה

ממשק החירום ששחזרתם במשימה 4 דורש שלוש פעולות בסדר מסוים: הכנת מצב חירום, בקשת אישור ולבסוף ביצוע. הבקר לא אמור לקבל בקשה שכוללת את הפעולות בסדר אחר, גם אם כולן מופיעות בה.

לפני שליחת הפקודה לביצוע, עליכם לבצע אימות אחרון: האם מדובר במכשיר הנכון? האם הבקשה אכן אושרה? האם האישור שייך לרכיב הנכון ולחשבון מוסמך? האם סדר הפעולות נשמר? **רק אם כל התנאים מתקיימים מותר להעביר את הפקודה לביצוע.**

הקבועים שב־`emergency.h`:

```
enum {
    STEP_PREPARE_EMERGENCY = 1,
    STEP_REQUEST_AUTHORIZATION = 2,
    STEP_EXECUTE_COMMAND = 3
};
#define M5_ADMIN_CLEARANCE 4
```

ממשו:

```
int validate_emergency_request(const EmergencyRequest *request);
```

חוזה הפונקציה:

- החזירו `1` רק עבור בקשה תקינה במלואה, ואחרת `0`. אין לשנות את הבקשה או לשחרר את הזיכרון שלה.
- שם המכשיר חייב להיות `BEERSHEVANATOR`, המצב `emergency` והפעולה `self-destruct`.
- מזהי הבקשה והרכיב שבבקשה ומזהה המאשר ברשומת האישור חייבים להיות חיוביים. `authorization.request_id` ו־`authorization.unit_id` חייבים להתאים למזהי הבקשה והרכיב שבמבנה. `approved` חייב להיות `1`, ו־`clearance` בטווח `M5_ADMIN_CLEARANCE` עד 5.
- `steps` חייב להצביע למערך בן **שלושה** צעדים, לפי הסדר: `STEP_PREPARE_EMERGENCY`, אחריו `STEP_REQUEST_AUTHORIZATION`, ולבסוף `STEP_EXECUTE_COMMAND`.
- דחו `NULL`, מצביעים או מחרוזות חסרים, ומספר צעדים שאינו שלושה. אין לקרוא מעבר לגבולות מערכים.

בדקו באמצעות `make test-d`. הפונקציה מאמתת את הבקשה בלבד; היא אינה מעניקה אישור ואינה מבצעת את ההשמדה בעצמה.

הפעלת המערכת

ב־`main.c` מסופקת תשתית לקריאת תיק הראיות ולהפעלת הבקר. בין הכנת הבקשה לבין אימותה תמצאו קריאה מוכנה ל־`mission5_authorize_via_port`, המאפשרת לנסות לקבל את האישור החסר דרך `PORT_8080`. הפונקציה ממומשת עבורכם; אין צורך לשנות אותה. השלימו רק את החלקים המסומנים, כך שהתוכנית תבצע את הרצף הבא:

1. קבלת הוראה תקינה משורת הפקודה.
2. איתור רשומה ממתינה שמתאימה לבקשה ולרכיב המופיעים בתיק הראיות.
3. הכנת בקשה עצמאית מהרשומה ומהצעדים ששוחזרו.
4. ניסיון לקבל אישור באמצעות הפונקציה המוכנה ב־`main.c`.
5. אימות הבקשה לאחר האישור והעברתה לבקר רק אם היא תקינה.
6. שחרור בטוח של הזיכרון בכל מסלול הצלחה או כישלון.

אין לקבע בקוד מזהים אישיים, לשנות ידנית את `approved` כדי לעקוף את שירות האישורים או להדפיס הצלחה במקום להפעיל את המערכת. בדיקות השלבים אינן מחליפות את השלמת `main.c`.

הריצו מתוך `/mission5`:

```
make run
```

רק הרצה מלאה מוצלחת תציג את תוצאת המבצע ואת קוד ההשלמה בפורמט `B7-M5-XXXXXXXX`.

דוח החקירה

מלאו את חלק משימה 5 ב־`investigation_report.md`, לאחר שהשלמתם את המשימה והרצתם אותה בהצלחה.

ענו **בקצרה ובמילים שלכם**. אין צורך לתאר מחדש את הקוד או להעתיק את הוראות המשימה. אין להדביק את פלט התוכנית המלא.

ממצאים

מה גיליתם על תהליך ההרשאה, מה הייתה תוצאת המבצע ומה נשאר לא ידוע בסיום החקירה? ציינו את קוד ההשלמה בפורמט `B7-M5-XXXXXXXX`.

התבססו רק על המידע שנחשף בהרצה.

הבנה טכנית

1. בשלב הכנת בקשת החירום, רשומת האישור מועתקת לתוך המבנה `EmergencyRequest`. מדוע שינוי הרשומה בתוך הבקשה לא משנה את הרשומה המקורית במאגר הראיות?
2. אילו נתונים בבקשה חייבים להישמר בעותק עצמאי, ומה עלול לקרות אם נשמרים רק מצביעים למאגר הראיות?
3. בחרו בדיקת תקינות אחת מתוך `validate_emergency_request`. הסבירו מה היא בודקת ומה עלול לקרות אם מוותרים עליה.

תשובת הצוות:

הגשה

הגישו:

- `mission5/emergency.c`
- `mission5/main.c`
- `investigation_report.md` המעודכן

אין לשנות או להגיש את תשתיות `/support/mission5_/..`, את קובץ הראיות או קובצי הרצה.